Fair Price in a Moving Market: Predicting Residential Prices

Phase III: Pilot Study of the Supervised Models

Zeyad Khaled
The American University in Cairo
Cairo, Egypt

Mohamed Akram
The American University in Cairo
Cairo, Egypt

ABSTRACT

We compare the supervised models covered in the course on the Zillow property file for Los Angeles, Orange and Ventura counties, predicting the natural log of each home's 2016 assessed value. Before training anything we rebuilt the dataset to answer the Phase 1 and Phase 2 feedback, and Section 1 maps each comment to the change it caused. The rebuilt table holds 2,859,302 homes described by 17 features, which expand to 606 model columns. Every model followed one protocol: a five-fold search for its settings on 2,287,442 training homes, one final fit on all of them, and a single score on 571,860 test homes that played no part in any choice. The regression models were also scored as five-band price classifiers, next to logistic regression, the perceptron and naive Bayes. On the test homes the neural network reached $R^2 = 0.467$ on the log value with a median miss of 32.5%, against 0.436 and 33.8% for linear regression. The three strongest models were **X**, **X** and **X**. We choose **X** [provisional: the neural network, to be confirmed when every model has finished].

KEYWORDS

model comparison, cross-validation, regression, classification, property valuation

1 RESPONSE TO THE PHASE 1 AND PHASE 2 FEEDBACK

Both earlier phases lost marks, and every comment pointed at something real. We rebuilt the dataset from the raw file before running a single model, so that the comparison in this report rests on data we can defend. Table 1 pairs each comment with what we changed and the measurement behind the change. Appendix A lists all twelve fixes in full.

2 DATA

2.1 The file and the label

The data is the `properties_2017` file from the Zillow Prize competition [28]: 2,985,217 parcels in three Southern California counties, described by 58 columns. The label is `taxvaluedollarcnt`, the county's 2016 assessed value of the parcel. The file contains no sale price, so the project predicts what the assessor wrote down, which is not always what the home would sell for (Section 8). We treat value as the sum of what a home's characteristics are worth, the hedonic view of housing [21].

Values are heavily skewed. The median training home is assessed at $324,441 while the largest reaches $88 million, and the raw label has a skew of 18.9 (Figure 1). We predict the natural log instead. Its skew is −0.12, so a few mansions no longer dominate the squared error, and a miss on the log scale behaves like a percentage miss: being 10% off counts the same on a $150,000 condominium as on a $3 million house. Every dollar figure in this report converts the log prediction back with the exponential.

2.2 Which homes are kept

Phase 2 kept every row that had a label, which let in shops, parcels with no building and assessed values of one dollar. We now keep only ordinary homes with a building on them. The rules in Table 2 run in order and each removed home is counted once, under the first rule it fails. The kept property types are single family, condominium, duplex, triplex, quadruplex, planned unit development, mobile home, cluster home, residential general, cooperative, manufactured home and townhouse.

2.3 Training homes, test homes and folds

The kept homes were shuffled with seed 42 and split 80/20: 2,287,442 training homes and 571,860 test homes. Each parcel appears exactly once in the file, so no home can sit on both sides of the split. The test homes were sealed. No setting, feature or rule was chosen by looking at them, and each final model was scored on them once.

The training homes were divided into five folds of about 457,000 homes each (457,575, 456,676, 457,411, 458,657 and 457,123), using a hash of the parcel number so that a home always lands in the same fold.

2.4 Features

The 17 features fall into three kinds.

Table 1: Each comment from the first two phases, what changed, and the number that supports the change. Association values are measured on training homes (Section 2.5).

Comment (phase)	What we changed	Evidence
Dataset too small; limits of the data not discussed (Phase 1)	The project moved to the Zillow file in Phase 2. This report states the limits of its label in Section 8.	2,985,217 parcels in the raw file; 2,859,302 homes kept.
What is FIPS? (Phase 2)	FIPS is the Federal Information Processing Standard county code. It is now read as a county name (6037 Los Angeles, 6059 Orange, 6111 Ventura). Its copy `regionidcounty` was removed with a measured reason.	Cramér's V of 1.000 between the two columns. The county itself was later dropped at 0.998 with the county land-use code.
Raw latitude and longitude used as inputs (Phase 2)	Coordinates became miles north and east of the median training home, since a degree of longitude here is 57 miles and a degree of latitude 69. Both were then dropped by the pair rule because ZIP code carries the same information.	0.983 and 0.988 with ZIP code; ZIP code is 0.490 with the label against 0.119 and 0.013.
City, county and region dropped (Phase 2)	All five location columns were brought back and measured. ZIP code, neighbourhood and census tract are kept as one-hot groups. City and county were dropped by the measured pair rule, not by assumption.	325, 98 and 16 model columns. City is 0.921 with ZIP code and weaker with the label (0.391 against 0.490).
No distribution analysis; log taken with no analysis (Phase 2)	Figure 1 shows the label and the living area before and after the log, with the skew of each.	Label skew 18.9 falls to −0.12; living-area skew 262.1 falls to 0.39.
A decision with no number, for every feature (Phase 2)	Each of the 58 raw columns has a written decision with its blank share, its number of values, its strength with the label and its strongest partner.	17 features kept, six dropped by the pair rule, three dropped by name as leaks (Appendix B).
Text feature dropped as hard to encode (professor's rule on text)	Zoning is one-hot encoded, with codes under 0.1% of training homes grouped together.	Raw zoning is 0.411 with the label and only 0.505 with ZIP code; 120 model columns.
AI use not declared (Phases 1 and 2)	Section 10 states how AI was used, and every code file opens with the same statement.	Every script carries the statement.
References not cited (Phases 1 and 2)	Every method, library and data source is cited where it is used.	28 references.

Table 2: Row rules, applied in order.

Rule	Homes removed
No assessed value	34,266
Assessed in a year other than 2016	3,379
Assessed value under $10,000	14,505
Not a residential home, or no type	13,768
No living area recorded (no building)	25,926
Zero bedrooms or no bathroom	34,071
Homes kept	2,859,302

- **Seven numbers**: bathrooms, bedrooms, year built, log living area, log lot area, stories and fireplaces. Each is standardized with the training average and spread.
- **Two yes/no columns**: pool, and hot tub or spa.
- **Eight categories**, one-hot encoded: ZIP code, zoning, neighbourhood, county land-use code, census tract, building quality, heating and air conditioning. A value held by fewer than 0.1% of training homes joins an "other" value, a blank becomes its own value ("not recorded"), and the most common value of each group is dropped as the baseline.

Several columns are blank for a reason rather than at random. Los Angeles never records garage area, fireplaces or stories, while Orange and Ventura never record building quality or zoning. A blank fireplace count in Orange means no fireplace; in Los Angeles it means nobody counted. We

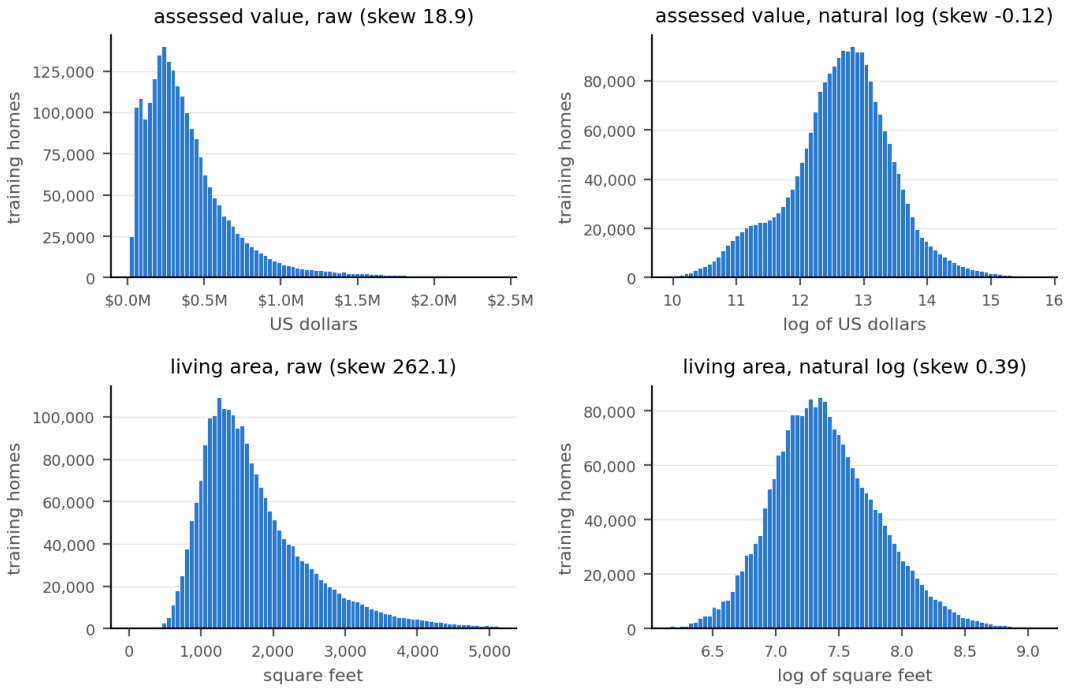


Figure 1: The assessed value (top) and the living area (bottom) of the 2,287,442 training homes, raw and after the natural log. Raw panels stop at the 99th percentile ($2.42 million and 5,120 square feet) so the bars stay visible; log panels drop the outer 0.1% on each side. Each skew is computed on every training home.

handled each column by what its blanks mean (Appendix A, Fix 6), and the ZIP code and land-use columns tell the models which county a home is in.

The census tract comes from the raw tract-and-block number with the block digits removed. Every list of values and every fill-in value was learned from the training homes only.

2.5 Correlation checks

The association between two columns was measured by the Pearson correlation for two numbers, by the correlation ratio for a category and a number, and by the bias-corrected Cramér's V for two categories [1, 8]. The correlation ratio of a many-valued category is inflated when it is measured on the homes it was fitted on, so it was measured on fold 0 after fitting the category averages on the other four folds. Figure 2 in the appendix shows every pair.

Four rules then decided which features stay.

(1) No feature may reach 0.85 with the label (the professor's rule). None did; the strongest is the log living area at 0.525.
(2) When two features reach 0.85 with each other, the one weaker with the label is dropped, strongest pair first. Six were dropped: county (0.998 with the county land-use code), miles east (0.988 with ZIP code), miles north (0.983 with ZIP code), property type (0.935 with the land-use code), city (0.921 with ZIP code) and garage area (0.906 with the land-use code).
(3) A dropped column comes back if it lies between 0.60 and 0.80 with the label and stays under 0.85 with every kept feature. Only the three leak columns sat in that window, and they are barred (below), so nothing came back.
(4) After one-hot encoding, the same pair rule ran on the model columns and removed 28 of them, mostly a neighbourhood or census tract that covers exactly the same homes as one ZIP code. The smallest eigenvalue of the column correlation matrix is 0.0041, so no column is an exact combination of the others.

Three columns were removed by name before any rule ran: the building value, the land value and the tax amount. The first two add up to the label exactly and the tax bill is about 1% of it. On the log label they measure only 0.583 to 0.653, under the 0.85 limit, because they are in dollars and the label is a log. The rule alone would have let them in.

3 EXPERIMENTAL SETUP

3.1 One protocol for every model

Every model went through the same four steps.

(1) **Search.** Each setting in the model's grid trains on four folds and is scored on the fifth, five times, so every training home is used for validation exactly once [16].
(2) **Winner.** The setting with the lowest average validation mean squared error (MSE) of the log value wins, counting only settings scored on all five folds.
(3) **Final fit.** The winning setting is trained once on all 2,287,442 training homes.
(4) **Test.** The final model is scored once on the 571,860 test homes.

The winner is picked on MSE of the log because most of the models minimise squared error during training, and on the log scale it measures relative misses. Linear regression has no setting to search, but it still ran on all five folds so its validation score is comparable. The two baselines were validated on fold 0 only.

3.2 Scores

For each model we report the MSE and the coefficient of determination (R^2) on the log value, R^2 and the root mean squared error (RMSE) in dollars, the median miss as a percentage of the true value, and the share of homes predicted within 20% of their value. R^2 in dollars is pulled around by the most expensive homes, which is why the selection works on the log.

3.3 Price bands for the classification models

The brief asks a regression project to also try classifiers on a discretised label. We cut the log value into five bands of equal size at the training quintiles. The edges fall at \$165,080, \$269,041, \$387,279 and \$580,090, and each band holds about 457,400 training homes. Logistic regression, the perceptron and naive Bayes were trained on these bands with the same four steps, choosing the winner by the highest average validation accuracy. Each regression model's test predictions were also cut at the same edges, so every model appears in one band table. Band results report accuracy, the macro-averaged F1 score (equal weight to each band) and the share of homes placed within one band of the true band.

3.4 Hardware and software

Everything ran on an Apple M1 Pro laptop with 16 GB of memory. The code is Python 3.12 with scikit-learn 1.8 [20] and PyTorch 2.11 [19], which ran the neural network and the nearest-neighbour search on the laptop's graphics processor. Every random step uses seed 42 (or 42 plus the fold number), so a rerun repeats the same splits, samples and starting weights.

The memory limit shaped several grids. A fully grown regression tree on 1.8 million homes has two to three million nodes, so a forest of 50 such trees would not fit in 16 GB. The grids in Table 3 are therefore smaller than we would choose on a larger machine, and each limit is explained in the model's section.

4 THE MODELS ONE BY ONE

Each subsection gives the model's setup and why its settings were chosen, its validation and test results, and how well it fits this problem. Table 5 collects the test scores.

4.1 Baselines

The average baseline predicts the average training value in dollars for every home, and the median baseline predicts the median. They learn nothing about homes, so they mark the score a model must beat to have learned anything. On the test homes the median baseline misses by a median of 45.4% (MSE of the log 0.702) and the average baseline by 53.8% (0.828). The average baseline's R^2 on the log is negative (-0.18): the average dollar value sits above the typical home, because a few very expensive homes pull it up.

4.2 Linear regression

Ordinary least squares fits one weight per model column [13]. It has no setting to tune, so it ran on the five folds only to give a comparable validation score. The five fold scores sit between 0.3928 and 0.3952 (average 0.3943, spread 0.0010), and the test score is 0.3944, with $R^2 = 0.436$ on the log and a median miss of 33.8%. Thirty percent of test homes land within 20% of their value.

Linear regression cuts the median baseline's error by 44% using nothing but additive effects. On the log scale each column adds a fixed percentage to the value, so a ZIP code adds the same percentage to a small condominium as to a large house. The model cannot learn that the price of an extra square foot depends on where the home is. That limit is its main weakness here. In its favour, it fits in seconds and every weight can be read and checked.

4.3 Neural network

The network is a fully connected feed-forward network trained by backpropagation [23]: 606 inputs, two hidden layers with the ReLU activation, dropout after each hidden layer [25], and one output for the log value. Training minimises the squared error in batches of 1,024 homes, reshuffled every pass. A run stops when the validation error has

Table 3: Every model, the lecture it comes from, the settings searched, what was held fixed, and the setting that won on validation.

Model	Lecture	Settings searched	Held fixed	Winner
Average and median baselines	–	none	predict one number for every home	–
Linear regression	4	none	ordinary least squares, all 606 columns	–
Neural network	7	hidden layers 256–128, 128–64, 512–256; dropout 0 or 0.2; SGD at learning rate 0.1 or Adam at 0.001 (12 settings)	ReLU; batch 1,024; MSE loss; at most 100 passes; stop after 5 passes without improvement	256–128, Adam 0.001, dropout 0.2
K-nearest neighbours	3	k = 1, 3, 5, 10, 15, 20, 30, 50; Euclidean or cosine distance (16 settings)	exact search; prediction = mean log value of the k neighbours	X
Decision tree	9, 10	smallest leaf 1, 5, 20, 50, 200 homes	squared-error splits; no depth limit	smallest leaf 200
Random forest	10, 13	columns per split: square root (24), 10% (60), 33% (199)	50 trees; bootstrap samples; smallest leaf 5	X
Bagging	10, 13	none	a forest that may split on all 606 columns; 50 trees; smallest leaf 5	–
Support vector machine	12	penalty C = 0.01 or 1; margin width ϵ = 0 or 0.05 (4 settings)	linear kernel; ϵ-insensitive loss; at most 2,000 passes	X
AdaBoost.R2	13	tree depth 4 or 8	50 rounds; linear loss	X
K-means clustering	14	50, 200 or 1,000 clusters	mini-batches of 4,096; 3 restarts; home priced at its cluster's median	X
Hierarchical clustering	14	50, 200 or 1,000 clusters	Ward linkage on 10,000 sampled homes; home joins nearest cluster centre	X
Voting	13	none	average of the three regression models with the best validation scores	X, X, X
Logistic regression	5	penalty C = 0.01, 1, 100	five bands; L2 penalty; at most 300 passes	X
Perceptron	6	5 or 20 passes	five bands, one against the rest; no penalty	X
Naive Bayes	11	smoothing 0.1, 1, 10	five bands; normal model for numbers, yes/no model for 0/1 columns	X

not improved for five passes, at most 100, and keeps the weights from its best pass.

The grid crosses three sizes with two dropout rates and two optimisers. Plain stochastic gradient descent (SGD) ran at a learning rate of 0.1, and Adam [15] at 0.001, the rate its authors recommend. Twelve settings on five folds made 60 runs, which took 3.5 hours on the laptop's graphics processor. Table 4 shows every setting.

Dropout decided the ranking more than size did. With Adam, adding dropout moved every size from last place to first, most likely because Adam without it fits the training folds quickly and then starts to memorise them. Those runs reached their best around pass 20 and did not improve after it. With SGD at 0.1 the noisy updates already act as a brake, and dropout changed little. The top three settings differ by 0.0007, less than the spread between folds, so they are effectively tied, and we took the listed winner without reading more into the order.

The final network (256–128, Adam, dropout 0.2) trained from fresh weights on all training homes for 27 passes, the median best pass of its five runs, in 4.5 minutes. On the test homes it scored an MSE of 0.3725 on the log (R^2 = 0.467), a median miss of 32.5%, 31.6% of homes within 20%, and R^2 = 0.569 in dollars with an RMSE of $405,643. That is 5.6% lower error than linear regression on the log scale.

The network can combine location with size and age, which linear regression cannot, and it trains on all 2.3 million homes in minutes because it reads them in batches. It is

Table 4: Neural network search: average over the five validation folds, best first. Spread is the standard deviation of the five fold scores.

Setting	MSE (log)	Spread	R^2 (log)	Median miss %	Best pass
256–128 Adam, drop 0.2	0.3730	0.0010	0.467	32.4	27
128–64 Adam, drop 0.2	0.3731	0.0011	0.467	32.4	38
512–256 Adam, drop 0.2	0.3737	0.0014	0.466	32.5	27
512–256 SGD, drop 0.2	0.3759	0.0033	0.463	32.8	39
256–128 SGD, drop 0	0.3760	0.0029	0.463	32.9	14
256–128 SGD, drop 0.2	0.3760	0.0026	0.463	32.7	28
512–256 SGD, drop 0	0.3765	0.0025	0.462	33.0	18
128–64 SGD, drop 0	0.3766	0.0028	0.462	33.0	29
128–64 SGD, drop 0.2	0.3770	0.0019	0.461	32.7	31
128–64 Adam, drop 0	0.3788	0.0020	0.459	33.0	19
256–128 Adam, drop 0	0.3790	0.0008	0.459	32.7	21
512–256 Adam, drop 0	0.3798	0.0006	0.457	33.0	20

harder to explain to a homeowner than a list of weights, it depends on the optimiser and learning rate being right, and runs on the graphics processor can differ in the last digits between reruns.

4.4 K-nearest neighbours

A home is priced at the mean log value of the k training homes closest to it in the 606-column space [6]. Training only stores the homes, so all eight values of k were scored from one neighbour search per distance. The search is exact, computed on the graphics processor in batches of 204 query homes against the 1.8 million homes of each fitting fold. Cosine distance compares the direction of two homes' columns and ignores their length, which is a different idea of "similar" from Euclidean distance; both were tried.

The best setting was **X** with a validation MSE of **X**, and the final model scored **X** on the test homes (median miss **X**%). [Comment on the result once known: whether small k copied label noise, and how the one-hot columns shaped the distances.] The model is easy to explain, since each prediction can be traced to real neighbouring homes. It is slow to query, though, because each new home is compared with every training home, and every one-hot column counts as much as the living area in the distance.

4.5 Decision tree

A regression tree splits the homes in two again and again, choosing each split to cut the squared error most, and predicts the average log value of the homes in a leaf [4]. We controlled its size by the smallest number of homes a leaf may hold. Pruning a grown tree by cost complexity would have been closer to the lecture, but a single full tree here takes about three minutes and the pruning grid would have taken days.

Leaf size mattered a great deal. With one home per leaf the tree memorised the training folds and scored 0.724 on validation, nearly twice the error of linear regression. A minimum of 5 homes gave 0.531, a minimum of 20 gave 0.433, and minimums of 50 and 200 gave 0.4137 and 0.4136, tied within the fold spread of about 0.0015. The final tree (smallest leaf 200), fitted on all training homes in 15.6 minutes, scored 0.4109 on the test homes ($R^2 = 0.412$ on the log) with a median miss of 34.7%.

A tree reads like a set of rules a valuer would recognise, and it needs no scaling. Here, though, a single tree is beaten by linear regression, because it gives every home in a leaf the same value and so predicts in steps. Its main role in this study is as the building block of the forest, bagging and AdaBoost.

4.6 Random forest

A random forest averages many trees, each grown on a bootstrap sample of the homes and allowed to split on only a random share of the columns at each split [3]. We searched that share (24, 60 or 199 of the 606 columns) and fixed 50 trees with at least 5 homes per leaf. One forest tree grown to single-home leaves on these homes has about two million nodes; with 5 homes per leaf it has about 276,000, which is what lets 50 of them fit in memory.

The best share was **X** (validation MSE **X**), and the final forest scored **X** on the test homes with a median miss of **X**%. [Comment once known: did averaging fix the single tree's overfitting, and which share won.] Forests are strong on tabular data and rarely need careful tuning. They are large in memory, slow to train on this many homes, and much harder to read than one tree.

4.7 Bagging

Bagging is the same forest with every column available at every split [2], so the trees differ only through their bootstrap samples. Comparing it with the random forest shows how much the random column choice adds. Bagging scored **X** on validation and **X** on the test homes. [Comment once known: whether bagging's trees were too alike compared with the forest's.]

4.8 Support vector machine

A linear support vector machine for regression ignores misses smaller than a margin ϵ and penalises larger ones in proportion to their size, weighted by C [5]. A kernel version would need a comparison between every pair of the 1.8 million homes, so we used the linear solver built for large data [14]. We searched C (0.01 or 1) and ϵ (0 or 0.05 on the log

scale, where 0.05 means misses under about 5% cost nothing). The best setting was **X** with validation MSE **X**, and the test score was **X**. [Comment once known: a linear model with absolute-error loss should land close to linear regression; say whether it did.]

4.9 AdaBoost

AdaBoost.R2 [11, 12] fits trees one after another, drawing each round's sample so that the homes earlier rounds missed most appear more often, and combines the rounds by a weighted median. We ran 50 rounds and searched the depth of each tree (4 or 8). The best depth was **X** (validation MSE **X**), and the test score was **X**. [Comment once known: boosting chases the hardest homes, and in this file many of those are homes whose assessed value lags their market value, so it may chase noise.]

4.10 K-means clustering

Clustering is unsupervised, so we turned it into a predictor: the training homes are grouped into k clusters without looking at the label, and a home is priced at the median log value of the training homes in its cluster. K-means [17] ran in mini-batches of 4,096 homes [24], which keeps it fast on 1.8 million rows, with three restarts. The best number of clusters was **X** (validation MSE **X**), and the test score was **X**. [Comment once known: the clusters follow ZIP code and size, which the label also follows, but each cluster gets one price.]

4.11 Hierarchical clustering

Ward's agglomerative method [27] merges the two clusters whose merge adds the least spread, starting from single homes. It needs the distance between every pair of homes, about 1.7×10^{12} pairs for all the training homes, so we built the hierarchy on a random sample of 10,000 and then assigned every home to the nearest cluster centre. Prices come from cluster medians, as in K-means. The best number of clusters was **X** (validation MSE **X**), and the test score was **X**. The sample is the obvious weakness: a cluster structure learned from about half a percent of the homes may miss small expensive neighbourhoods.

4.12 Voting

Voting averages the log predictions of the three regression models with the lowest average validation MSE [9]. The members were picked on validation scores only, so the test homes still played no part. The members were **X**, **X** and **X**, and the vote scored **X** on the test homes. Voting has no validation score of its own, so it cannot be ranked against the other models on validation and is reported on the test homes only. [Comment once known: averaging helps most when the members make different mistakes.]

4.13 Price-band classifiers

Logistic regression [7] models the probability of each band as a softmax of a linear score, with an L2 penalty whose strength is set by C; we searched C = 0.01, 1 and 100. The perceptron [22] learns one linear boundary per band against the rest, and we searched 5 or 20 passes. Naive Bayes [10] treats the columns as independent within a band; it uses a normal distribution for each of the seven standardized numbers and a yes/no probability for each 0/1 column, with smoothing 0.1, 1 or 10.

The winners were **X**, **X** and **X**. On the test homes they placed **X**%, **X**% and **X**% of homes in the right band (macro F1 **X**, **X**, **X**). Guessing the most common band would score about 20%. [Comment once known: compare with the regression models cut into bands in Table 6; the independence assumption of naive Bayes is badly broken by groups such as ZIP code and neighbourhood, which describe the same place.]

5 ALL MODELS SIDE BY SIDE

Table 5 ranks the regression models by validation and shows their test scores next to it. Table 6 shows the same comparison as a five-class problem. [Two or three sentences once the tables are full: which family won, how far the best model is from linear regression, and whether the band table agrees with the regression ranking.]

6 THE TOP THREE MODELS

Ranked by average validation MSE of the log, the three strongest models are **X**, **X** and **X** (Table 7). [At the time of writing, the neural network leads with 0.3730.]

6.1 Where the variability comes from

Between folds. The five fold scores of one setting vary very little: the spread is 0.0010 for linear regression, between 0.0006 and 0.0033 for the network settings, and between 0.0011 and 0.0021 for the decision trees. Each fold holds about 457,000 homes, so a fold's average error is already a very stable number. The practical rule we draw from this is that two settings closer than about 0.002 are tied. The three best network settings fall inside that margin, and so do the decision trees with leaves of 50 and 200 homes. SGD runs varied more than Adam runs (0.0019 to 0.0033 against 0.0006 to 0.0014), and their best pass ranged from 14 to 39. A large fixed learning rate makes the error bounce from pass to pass, so where a run stops depends on luck.

Between validation and test. The test scores agree with validation to within a fold spread: 0.3730 against 0.3725 for

Table 5: Every regression model on the 571,860 test homes, ranked by average validation MSE of the log (the baselines on fold 0 only). Each model was scored on the test homes once.

Model	Validation MSE (log)	Test MSE (log)	Test R^2 (log)	Test R^2 ($)	RMSE ($)	Median miss (%)	Within 20% (%)
Neural network	0.3730	0.3725	0.467	0.569	405,643	32.5	31.6
Linear regression	0.3943	0.3944	0.436	0.490	441,385	33.8	30.0
Decision tree	0.4136	0.4109	0.412	0.454	456,740	34.7	29.6
Random forest	X	X	X	X	X	X	X
Bagging	X	X	X	X	X	X	X
Support vector machine	X	X	X	X	X	X	X
AdaBoost.R2	X	X	X	X	X	X	X
K-nearest neighbours	X	X	X	X	X	X	X
K-means clustering	X	X	X	X	X	X	X
Hierarchical clustering	X	X	X	X	X	X	X
Voting (top three)	–	X	X	X	X	X	X
Median baseline	0.7011	0.7021	−0.004	−0.036	629,192	45.4	22.3
Average baseline	0.8261	0.8278	−0.184	0.000	618,192	53.8	20.9

Table 6: Every model on the five price bands of the test homes. Band models were trained on the bands; regression models had their predictions cut at the same edges.

Model	Accuracy (%)	Macro F1	Within one band (%)
Trained on the bands			
Logistic regression	X	X	X
Perceptron	X	X	X
Naive Bayes	X	X	X
Regression models, cut into bands			
Neural network	X	X	X
Linear regression	X	X	X
[every other regression model]	X	X	X

Table 7: The three strongest models. Spread is the standard deviation of the five validation folds; county and band rows give the test median miss.

	X	X	X
Validation MSE (log)	X	X	X
Fold spread	X	X	X
Test MSE (log)	X	X	X
Test median miss (%)	X	X	X
Los Angeles	X	X	X
Orange	X	X	X
Ventura	X	X	X
cheapest band	X	X	X
dearest band	X	X	X
Final training time	X	X	X

the network, 0.3943 against 0.3944 for linear regression. [add the other top-three models]. The test homes were never used to choose anything, so this agreement suggests the search did not overfit the folds. The decision tree did slightly better on the test homes than on validation (0.4109 against 0.4136). Its final fit saw all 2.3 million training homes instead of the 1.8 million of a fold, and with a fixed minimum of 200 homes per leaf, more homes allow more and finer leaves.

Between models. The differences between models are larger than the differences between folds, and they follow how each model handles noise. The decision tree with one home per leaf copies each training home's label, including its noise, and its error nearly doubles. Averaging (forest, bagging, voting) and larger leaves smooth that noise away. [expand with the forest and K-nearest-neighbour results.]

Between parts of the market. [Fill from Table 7: whether the top models miss more in one county or one price band. Los Angeles never records garage area, fireplaces or stories, so homes there are described by fewer columns.]

The ceiling set by the label. Every model stops well short of a perfect fit, and the gap between the best model and linear regression is small (0.0219 on the log scale). We think most of the remaining error belongs to the label rather than to the models. Under Proposition 13 a California home's assessed value is set at its purchase price and can then rise by at most 2% a year until the home sells again [26]. Two identical neighbouring houses, one bought in 1980 and one in 2015, can carry assessed values several times apart. The file does not record when a home last sold, so no feature can explain that gap, and every model meets the same floor.

7 CHOICE OF MODEL

[Provisional. To be confirmed once every model has finished.] We choose the neural network: two hidden layers of 256 and 128 units, Adam at a learning rate of 0.001, dropout 0.2, trained for 27 passes.

Performance. It has the lowest validation error so far (0.3730) and kept it on the test homes (0.3725), with a fold spread of 0.0010. **X** [compare with the second and third models].

Nature of the model. A network can learn that the value of a square foot depends on the ZIP code and the age of a home. Linear regression has to add those effects separately. A network also does it without a hand-built list of interactions.

Fit to the problem. The table is large (2.3 million training homes) and wide (606 columns, mostly one-hot). The network reads it in batches, so its memory use stays flat as the data grows. It trained on every home in 4.5 minutes and prices a new home in a fraction of a second, which suits the service planned for Phase V. Its weakness is explanation: a valuer cannot read why it gave a value, so for that purpose linear regression stays as the reference.

8 LIMITATIONS

- **The label is an assessed value, not a sale price.** Proposition 13 ties it to the purchase price at the last sale [26]. The competition's own target, the error of Zillow's estimate against the sale price, was withheld, so the file contains no market price at all.
- **Fine location is lost.** The raw census block (about 94,000 values) is 0.592 with the label on held-out homes, stronger than ZIP code. One-hot encoding at block level would need about 94,000 columns. Target encoding [18] could carry it, but we did not use it in this phase.
- **Garage area was dropped** at 0.906 with the county land-use code. That value comes mostly from Los Angeles never recording garages, so the rule removed a column that does carry information in Orange and Ventura.
- **Implausible living areas remain**, from 1 square foot to 952,576. The log limits their pull, but we did not clip them.
- **Smaller grids than ideal.** Forests use at least five homes per leaf and 50 trees, the single tree is pre-pruned, and hierarchical clustering uses a sample of 10,000 homes, all because of the 16 GB laptop.

Table 8: The scripts and what each one runs.

Script	Runs
`zillow_columns.py`	the decision and reason for each of the 58 raw columns
`zillow_rows.py`	row rules, the 80/20 split, the five folds
`zillow_features.py`	the 17 features
`zillow_checks.py`	association tables, heatmaps, the pair rules
`zillow_table.py`	encoding, scaling, the 606-column table
`train_models.py`	baselines, linear regression
`neural_search.py`	neural network five-fold search
`final_training.py`	final network, test scores
`slide_models.py`	the four-step runner, K-nearest neighbours, support vector machine
`zillow_models.py`	decision tree, random forest, bagging, AdaBoost, both clusterings
`band_models.py`	logistic regression, perceptron, naive Bayes
`zillow_results.py`	voting and the result tables
`report_figures.py`	Figure 1
`test_zillow.py`	seven unit tests
`run_all.sh`	every step in order

9 SOURCE CODE

The scripts are submitted with this report. Table 8 lists what each one runs; `run_all.sh` runs every step in order, and each step skips work it has already finished.

10 USE OF AI

[Check that this statement matches what you did before submitting.] We used Claude (Anthropic), an AI assistant, to help write and debug the code and to draft parts of this report. We set the rules, chose or approved every setting, ran every experiment, checked the reported numbers against the output files, and edited the text. Every code file opens with the line: "AI was used to help write this code, but the resulting code was reviewed and edited by a human."

REFERENCES

[1] Wicher Bergsma. 2013. A Bias-Correction for Cramér's V and Tschuprow's T. *Journal of the Korean Statistical Society* 42, 3 (2013), 323–328. https://doi.org/10.1016/j.jkss.2012.10.002

[2] Leo Breiman. 1996. Bagging Predictors. *Machine Learning* 24, 2 (1996), 123–140. https://doi.org/10.1007/BF00058655

[3] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32. https://doi.org/10.1023/A:1010933404324

[4] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. 1984. *Classification and Regression Trees.* Wadsworth.

[5] Corinna Cortes and Vladimir Vapnik. 1995. Support-Vector Networks. *Machine Learning* 20, 3 (1995), 273–297. https://doi.org/10.1007/BF00994018

[6] Thomas Cover and Peter Hart. 1967. Nearest Neighbor Pattern Classification. *IEEE Transactions on Information Theory* 13, 1 (1967), 21–27.

https://doi.org/10.1109/TIT.1967.1053964

[7] David R. Cox. 1958. The Regression Analysis of Binary Sequences. *Journal of the Royal Statistical Society, Series B* 20, 2 (1958), 215–242.

[8] Harald Cramér. 1946. *Mathematical Methods of Statistics*. Princeton University Press.

[9] Thomas G. Dietterich. 2000. Ensemble Methods in Machine Learning. In *Multiple Classifier Systems (Lecture Notes in Computer Science, Vol. 1857)*. Springer, 1–15. https://doi.org/10.1007/3-540-45014-9_1

[10] Pedro Domingos and Michael Pazzani. 1997. On the Optimality of the Simple Bayesian Classifier under Zero-One Loss. *Machine Learning* 29, 2–3 (1997), 103–130. https://doi.org/10.1023/A:1007413511361

[11] Harris Drucker. 1997. Improving Regressors Using Boosting Techniques. In *Proceedings of the 14th International Conference on Machine Learning*. 107–115.

[12] Yoav Freund and Robert E. Schapire. 1997. A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. *J. Comput. System Sci.* 55, 1 (1997), 119–139. https://doi.org/10.1006/jcss.1997.1504

[13] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

[14] Chia-Hua Ho and Chih-Jen Lin. 2012. Large-Scale Linear Support Vector Regression. *Journal of Machine Learning Research* 13 (2012), 3323–3348.

[15] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *Proceedings of the 3rd International Conference on Learning Representations*. https://arxiv.org/abs/1412.6980

[16] Ron Kohavi. 1995. A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence*.

[17] Stuart Lloyd. 1982. Least Squares Quantization in PCM. *IEEE Transactions on Information Theory* 28, 2 (1982), 129–137. https://doi.org/10.1109/TIT.1982.1056489

[18] Daniele Micci-Barreca. 2001. A Preprocessing Scheme for High-Cardinality Categorical Attributes in Classification and Prediction Problems. *ACM SIGKDD Explorations Newsletter* 3, 1 (2001), 27–32. https://doi.org/10.1145/507533.507538

[19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32*. 8024–8035.

[20] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[21] Sherwin Rosen. 1974. Hedonic Prices and Implicit Markets: Product Differentiation in Pure Competition. *Journal of Political Economy* 82, 1 (1974), 34–55. https://doi.org/10.1086/260169

[22] Frank Rosenblatt. 1958. The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review* 65, 6 (1958), 386–408. https://doi.org/10.1037/h0042519

[23] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. 1986. Learning Representations by Back-Propagating Errors. *Nature* 323, 6088 (1986), 533–536. https://doi.org/10.1038/323533a0

[24] D. Sculley. 2010. Web-Scale K-Means Clustering. In *Proceedings of the 19th International Conference on World Wide Web*. 1177–1178. https://doi.org/10.1145/1772690.1772862

[25] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research* 15 (2014), 1929–1958.

[26] State of California. 1978. California Constitution, Article XIII A (Proposition 13, Tax Limitation).

[27] Joe H. Ward. 1963. Hierarchical Grouping to Optimize an Objective Function. *J. Amer. Statist. Assoc.* 58, 301 (1963), 236–244. https://doi.org/10.1080/01621459.1963.10500845

[28] Zillow. 2017. Zillow Prize: Zillow's Home Value Prediction (Zestimate). Kaggle competition, data file properties_2017.csv. https://www.kaggle.com/c/zillow-prize-1

A THE TWELVE FIXES SINCE PHASE 2

(1) **Only ordinary homes with a building.** The row rules in Table 2 replace keeping every labelled row; 125,915 rows were removed.

(2) **A new split and folds.** 2,287,442 training and 571,860 test homes; five folds by parcel number.

(3) **The label is the natural log** of the assessed value (Figure 1).

(4) **Location.** Raw coordinates became miles north and east, then gave way to ZIP code; city, ZIP code, neighbourhood and census tract were brought back and measured.

(5) **The county code** is read as a county name; its duplicate was dropped at Cramér's V 1.000.

(6) **Columns some counties never record.** Garage area and fireplaces: a blank becomes 0. Building quality, heating, air conditioning and zoning: a blank is its own value. Stories: the training median. Room count (Los Angeles writes 0 for every home) and unit count (recorded only in Los Angeles) were dropped.

(7) **Zoning kept**: 0.411 with the label, one-hot in 120 columns.

(8) **The county land-use code** is one-hot (19 columns) instead of frequency encoded, so two codes that happen to be equally common no longer share a number.

(9) **Columns Phase 2 left out** (pool, hot tub or spa, fireplaces, stories, air conditioning, zoning) were measured and kept where they earned it.

(10) **Duplicates dropped with a number**: `calculatedbathnbr` (1.000 with bathrooms), `fullbathcnt` (0.987), `finishedsquarefeet12` (1.000 with living area), the three pool-type columns (the same homes in 99.98% to 100% of cases).

(11) **Leak columns removed by name**: building value, land value and tax amount (0.583 to 0.653 with the log label).

(12) **The AI-use statement** is in every code file and in Section 10.

B THE FEATURES AND THEIR NUMBERS

Table 9: The 23 candidate features, by strength with the label on held-out homes. The last six were dropped by the pair rule.

Feature	With label	Strongest partner	Value	Model columns
Living area (log)	0.525	bathrooms	0.791	1
ZIP code	0.490	miles east	0.988	325
Bathrooms	0.485	living area	0.791	1
Building quality	0.434	garage area	0.763	10
Zoning	0.305	garage area	0.777	120
County land-use code	0.290	county	0.998	19
Neighbourhood	0.283	ZIP code	0.754	98
Year built	0.275	ZIP code	0.646	1
Bedrooms	0.261	living area	0.699	1
Heating	0.246	garage area	0.709	5
Pool	0.194	land-use code	0.717	1
Hot tub or spa	0.143	ZIP code	0.320	1
Census tract	0.141	ZIP code	0.533	16
Air conditioning	0.137	heating	0.429	4
Fireplaces	0.104	land-use code	0.643	1
Stories	0.081	ZIP code	0.513	1
Lot area (log)	0.073	land-use code	0.643	1
City	0.391	ZIP code	0.921	–
Garage area	0.174	land-use code	0.906	–
Miles north	−0.119	ZIP code	0.983	–
County	0.109	land-use code	0.998	–
Property type	0.073	land-use code	0.935	–
Miles east	−0.013	ZIP code	0.988	–

A feature's strongest partner can be a feature that was later dropped; the pair rule only removes the weaker side of a pair at 0.85 or more. Model columns add to 606 after the 28 columns removed by the column-level pair rule and the 8 baselines.

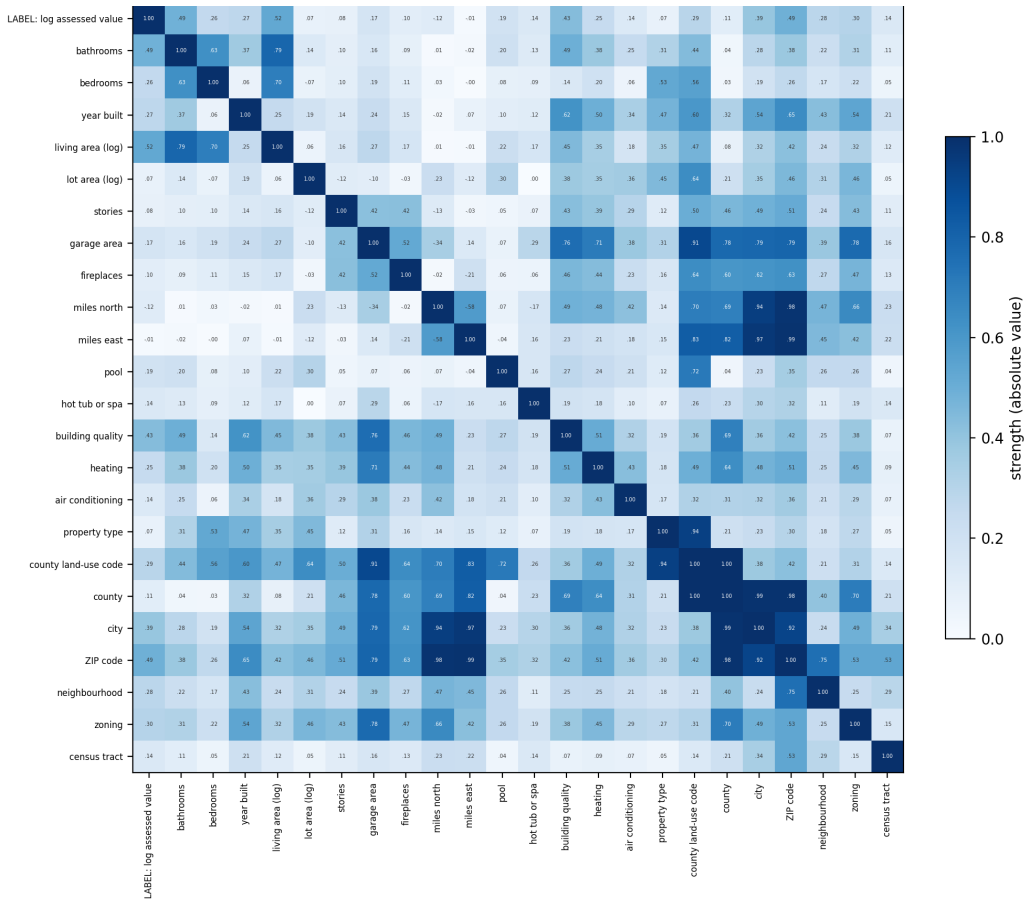


Figure 2: Association between the label and the 23 candidate features on the training homes: Pearson correlation for two numbers, correlation ratio on held-out homes for a category and a number, bias-corrected Cramér's V for two categories.